Local AI Lab

Part 18 - Questions for the Report

Answers based on the project implementation and the measurements recorded in Part 16.

Table of Contents

1. Foundations of Local AI . . . 3

Q1. What is Local AI? . . . 3

Q2. Difference between a Cloud LLM and a Local LLM? . . . 3

Q3. What role does Ollama play? . . . 3

2. Model Concepts . . . 3

Q4. What is model quantization? . . . 3

Q5. Why does model size affect memory consumption? . . . 3

Q6. Why can a smaller model sometimes be preferable? . . . 4

3. Benefits and Limits of Local Inference . . . 4

Q7. Advantages of local inference? . . . 4

Q8. Disadvantages of local inference? . . . 4

4. Project Architecture and Technologies . . . 4

Q9. Explain the architecture of your Python application. . . . 4

Q10. Explain the role of the REST API. . . . 5

Q11. What is RAG? . . . 5

Q12. Difference between an LLM and a multimodal LLM? . . . 5

5. Experimental Comparison and Security . . . 6

Q13. Compare two local models experimentally. . . . 6

Q14. Which model would you deploy on a computer with limited RAM? . . . 6

Q15. What security risks exist when exposing a local LLM server to a network? . . . 7

> **Overview.** The 15 questions are grouped into five themes: foundations of Local AI, model concepts, benefits and limits of local inference, the project architecture, and the experimental comparison with security considerations.

1 Foundations of Local AI

Q1. What is Local AI?

Local AI is artificial intelligence that runs directly on a local computer instead of relying on a remote cloud server. It uses local models and hardware to process user requests. In this project, **Ollama** runs the language model locally.

Q2. Difference between a Cloud LLM and a Local LLM?

Criterion	Cloud LLM	Local LLM
Where it runs	Remote servers, via internet or API	The user's own computer
Computing power	Generally greater	Limited by local hardware
Data control	Data leaves the machine	Greater control over data
Connectivity	Requires internet	Works offline once the model is installed

Q3. What role does Ollama play?

Ollama is the tool used to **download, manage, and run** language models locally. It provides a REST API that allows the Python application to send prompts to a model and receive generated responses.

2 Model Concepts

Q4. What is model quantization?

Model quantization reduces the precision used to represent a model's numerical parameters. For example, a model can use **4-bit instead of 16-bit** weights. This reduces memory consumption and can improve inference speed, although it may slightly reduce answer quality.

Q5. Why does model size affect memory consumption?

A model contains millions or billions of parameters that must be stored in memory during inference. Larger models generally require more RAM or GPU memory. Additional memory is also needed for:

- the input context,
- the generated tokens,
- intermediate computations.

Q6. Why can a smaller model sometimes be preferable?

A smaller model can run faster, consume less memory, and work on computers with limited hardware. It is often suitable for simple tasks and local applications.

> **Trade-off.** A smaller model may produce less accurate or less detailed answers for complex questions.

3 Benefits and Limits of Local Inference

Q7. Advantages of local inference?

- **Privacy:** better control over data.
- **Autonomy:** reduced dependence on internet connectivity.
- **Cost:** no per-request cloud API fees.
- **Flexibility:** greater control over the model and its configuration.
- **Infrastructure:** inference can run on personal or organizational hardware.

> **Note.** Local inference still consumes electricity and requires suitable hardware.

Q8. Disadvantages of local inference?

- **Speed:** can be slower than cloud inference when resources are limited.
- **Memory:** large models may require substantial RAM or GPU memory.
- **Quality:** answers can be of lower quality than those of more powerful cloud models.
- **Responsibility:** the user handles installation, maintenance, and security.

4 Project Architecture and Technologies

Q9. Explain the architecture of your Python application.

The application uses a Python backend connected to Ollama through a REST API. The user submits a question through a web interface built with Flask. The backend retrieves relevant information from local documents using a RAG component, then sends the prompt and retrieved context to the local language model through Ollama. The generated answer is returned and displayed with the response time and document sources.

Processing flow:

Main components:

Component	Role
Web interface	Receives user questions and displays answers
Flask backend	Handles requests and conversation logic
RAG component	Retrieves relevant information from local documents
Ollama	Manages communication with the local model
Local LLM	Generates the final answer
Performance measurements	Record response time and generation speed

Q10. Explain the role of the REST API.

A REST API allows different software components to communicate through HTTP requests. In this project, Python sends a **POST request** containing the model name and prompt to Ollama's API. Ollama processes the request and returns the generated response in **JSON** format. This lets the application use the model without implementing the inference engine itself.

Q11. What is RAG?

RAG stands for **Retrieval-Augmented Generation**. It combines information retrieval with text generation: before answering, the system searches a document collection for relevant information and adds the retrieved content to the prompt sent to the language model.

In this project, the RAG component retrieves relevant **.txt** documents from the local documents folder. This helps the model answer using the available course material and return “Information not available” when the required information cannot be found.

Q12. Difference between an LLM and a multimodal LLM?

A traditional text-based LLM processes and generates text. A multimodal LLM can process several types of input, such as text and images, and use them together to produce a response.

> **In the project.** The vision model **llava** analyzed an image of a cat and described the main objects, the environment, and the visible text, demonstrating how multimodal AI extends beyond text-only interaction.

5 Experimental Comparison and Security

Q13. Compare two local models experimentally.

In Part 16, **gemma3:latest** and **qwen3.5:latest** were compared using the same 500-word reinforcement-learning prompt. All measurements were collected on the author's own computer.

Model	Disk (GB)	Loaded (GB)	VRAM (GB)	Load (s)	Response (s)	Tokens	Tokens/s	RAM used (%)	Quality /5
gemma3:latest	3.35	2.41	0.26	41.59	188.37	808	5.65	69.1	Not rated
qwen3.5:latest	6.55	6.24	0.14	55.46	473.43	767	1.86	77.2	Not rated

Key findings

- **gemma3:latest** generated **5.65 tokens/s**, versus 1.86 tokens/s for qwen3.5:latest.
- Its total response time was substantially shorter (188.37 s versus 473.43 s).
- Its loaded size was smaller (2.41 GB versus 6.24 GB).

> **Limits of the benchmark.** The results are specific to this hardware, these model configurations, and this test prompt. They do not establish that one model is universally better, because answer quality was not quantitatively evaluated.

Q14. Which model would you deploy on a computer with limited RAM?

I would choose **gemma3:latest**, for three measured reasons:

- a smaller loaded size (2.41 GB versus 6.24 GB),
- faster responses,
- a lower measured system RAM usage (69.1% versus 77.2%).

It therefore appears more suitable for limited memory in this experiment. Before a real deployment, answer quality and memory usage should also be tested under the expected workload.

Q15. What security risks exist when exposing a local LLM server to a network?

Exposing a local LLM server can allow unauthorized users to access the API, consume computing resources, or reach sensitive data. Malicious documents may also manipulate the model through **prompt injection**.

Risk	Mitigation
Unauthorized API access	Restrict to trusted users; add authentication
Resource consumption	Firewall rules limiting reachable devices
Exposure of sensitive data	HTTPS/TLS when remote access is required
Prompt injection via documents	Control and review the documents used by the RAG component

> **Project note.** Ollama does not provide authentication by default. Binding it to **0.0.0.0** may expose the API to other devices on the network; a firewall and an authenticated reverse proxy with TLS can help limit and protect access.